有道翻译 输入arXiv论文检索地址，免费翻译全文→ https://fanyi.youdao.com/trans/#/home

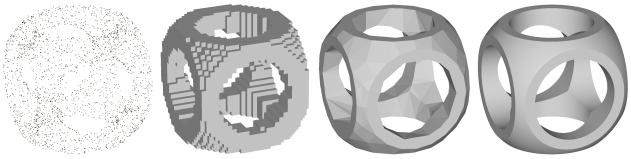
不要与我啮合：通过微调代码生成LLM来生成构造性实体几何而不是网格

Maximilian Mews, Ansar Aynetdinov, Vivian Schiller, Peter Eisert, Alan Akbik
Humboldt-Universität zu Berlin

Abstract

虽然机器学习的最新进展，如*llm*，正在彻底改变软件开发和创意产业，但它们对工程师设计机械部件的影响很小，这仍然主要是手工过程。现有的生成*3D*几何图形的方法最常用的是网格作为*3D*表示。虽然网格适用于视频游戏或动画中的资产，但它们缺乏足够的精度和适应机械工程目的。本文介绍一种新的*3D*几何生成方法，通过利用代码生成*LLM*来生成基于表面的构造性实体几何（*CSG*）。首先，通过将边界表示几何（*BREP*）转换为基于*csg*的*Python*脚本，创建一个以代码脚本表示的*3D*机械零件数据集。其次，我们使用*GPT-4*在自然语言中创建注释。结果数据集用于微调代码生成*LLM*。微调的*LLM*可以以合理的方式完成基于位置输入和自然语言的几何，展示几何理解。

1. 简介



(a) Point Cloud (b) Voxel Grid (c) Mesh (d) CAD/CSG

Figure 1. 3D几何图形的不同表示。

2023年1月，ChatGPT [27]上线仅两个月，月活跃用户就达到1亿，创造了用户增长最快的记录[14]。2024 Stack Overflow开发人员调查发现，62%的开发人员在他们的工作流程中使用AI工具，82%使用它们编写代码[39]。ChatGPT [27]和GitHub Copilot [9]等工具可以帮助开发人员根据文本输入生成或扩展代码，加快开发速度并扩展编码知识。机械工程师使用CAD软件设计3D零件，但这一过程仍然是手动的。人工智能驱动的CAD工具可以加速开发，甚至允许非专家，如产品所有者或经理，通过自然语言输入创建部件。

现有的3D几何生成方法要么只能随机生成机械部件，而不是文本控制的生成，要么以不精确的格式提供结果。例如，DreamFusion [30]和MeshGPT [37]等方法允许基于文本输入生成3D几何图形，但以神经辐射场的形式提供结果，这可以转换为网格。网格是由大量三角形的边连接而成的一种三维表示。也就是说，这种格式表示的所有非直线边和非平面表面都只是近似值。这导致了两个主要问题。

- 不精确 圆只能表示为多边形，圆形表面只能表示为分段平面三角形的组合，如Fig. 1所示。因此，由该方法生成的几何形状是不精确的，缺乏光滑的边缘和表面。
- 缺乏可修改性 由于所有的曲面都由三角形网格表示，模型不容易修改。例如，要改变圆柱体的大小，需要正确识别大量的三角形，然后移动和缩放。

机械零件大多是由基本的基元组合而成，如圆柱体、立方体或球体。通过一组有限的参数（如高度、半径和位置）直接表示*e.g.*圆柱体，比使用三角形网格逼近它要高效得多。该方法还能提高模型的精度，使模型修改更加容易。这种类型的三维表示称为边界表示（BREP），通常用于CAD软件。DeepCAD [46]是一种可以生成CAD序列的方法，但是它只能生成随机的机械部件，并且它没有办法通过输入（如自然语言）控制生成。到目前为止，Text2CAD [17]是唯一可以从文本描述生成CAD序列的方法。这是通过在Onshape [26]用户创建的基于草图的CAD序列的特定数据集上训练transformer架构来实现的。这意味着模型受到特定CAD软件数据的限制。

针对现有方法的不足，本文做出以下贡献：

1. 本文提出了一个数据集创建管道[1]，采用BREP几何形式。STEP文件并将其转换为Python脚本形式的基于表面的CSG序列，以便在任何现代CAD软件创建的3D文件上进行模型训练，而不仅仅是在shape上。
2. 用GPT-4o生成3D几何的自然语言描述，以便向训练数据集添加文本注释。
3. 训练了一个代码生成LLM，可以以合理的方式完成Python脚本表示的3D几何图形。此外，它能够利用所述3D几何体的自然语言描述来控制简单几何体的生成。

据我们所知，由此产生的模型是第一个专门训练以生成CSG几何的深度学习模型，也是第一个根据文本指令进行微调以完成3D几何的LLM。

[1]创建数据集的代码将在我们的Github仓库中提供：mxmws.github.io/dont_mesh_with_me

2. 相关工作

三维形状生成领域见证了旨在创建三维模型的各种计算策略[7, 30, 37**?**]。这些策略主要采用几何表示，如点云、体素网格或网格。尽管它们被广泛使用，但这些表示缺乏创建机械CAD部件所必需的精度和参数控制。

大型语言模型（llm）的微调以实现3D几何图形的生成在很大程度上仍未探索。

点云生成　点云生成已经通过GANs [36, 42]，可逆流[50]，梯度场[2]，自动编码器[51, 53]，以及最近的扩散模型[21, 25, 54]取得了进展，每个都提高了控制或表示的保真度。

点云方法擅长处理不规则结构和可变密度，但它们的离散性质损害了表面连续性，阻碍了高保真表示。它们缺乏点连通性也使表面生成等任务复杂化，在这些任务中，结构化表示被证明是有利的[51]。

体素生成　一些形状生成方法使用体素网格来表示3D几何[1, 5, 40, 45]。体素网格生成的一个显著进步是潜空间中模型变化的自动编码器的应用，如Schönhof *et al*. [**?**]所示。通过将体素表示编码到一个紧凑的潜空间中，自动编码器通过潜空间操作促进了不同体素化形式的生成。进一步扩展了生成能力，3D-GANs [44]（生成性对抗网络）已被用于从学习到的形状分布中生成体素网格。

体素图形可以导致3D空间中的“块状”外观，只能近似而不是完美地表示平滑的曲线和有机形状。

网格生成　生成模型领域通过操纵潜在空间在创建基于网格的几何图形方面取得了进展[4, 10, 12, 24]。Umetani [41]探索了自动编码器，以实现模型的变化。文本到网格生成领域已经取得了显著的进展，特别是通过利用隐字段表示进行网格提取的方法。突出的例子包括Liu等人的工作[20]和ShapeCrafter [7]，他们都利用自然语言输入来指导3D形状生成。相比之下，MeshGPT [37]采用基于transformer的架构直接生成3D网格，绕开了隐式场建模的需要。值得注意的是，MeshGPT能够产生具有尖锐和明确特征的几何图形。

这些方法擅长从文本创建网格，使它们适合在游戏和动画中使用。然而，由于网格划分的性质，生成的模型不能准确地表示圆形曲面，缺乏参数化，整体精度不高。

神经场生成　3D几何的神经表示，称为神经隐场或神经场，将物体或场景的几何编码为由神经网络表示的连续函数[23, 29]。在此基础上，一些工作探索了将神经领域用于生成任务，如生成新的3D形状[8, 30, 33]或现有场景的新视图[38]。这些生成式神经领域方法为灵活和可控的3D内容创造提供了潜力，在计算机图形学、虚拟现实和设计等领域都有应用。

虽然这些方法擅长表示光滑的表面并提供连续的表示，但它们通常与高频细节进行抗争，需要大量的计算资源来训练和推理，并且在编辑或操纵下一代时可能具有挑战性。

BREP世代　BrepGen [49]是最近的一种基于扩散的方法，可以直接生成CAD模型作为边界表示（BREPs）。它引入了一种层次树结构，其中每个节点编码几何基元，并通过节点复制隐式地获取拓扑结构。BrepGen生成适合CAD应用的3D几何图形。然而，它不能基于自然语言生成几何图形。

长方体生成　ShapeAssembly框架[15]通过使用名为“ShapeAssembly”的领域特定语言编写程序来运行，该语言旨在清晰地表达3D形状的组装。首先，通过将现有3D模型转换为ShapeAssembly语言来创建数据集，这反过来促进了深度生成模型的训练，特别是分层序列变分自编码器（VAE）。

ShapeAssembly的范围仅限于生成完全由长方体组成的结构。

使用**llm**生成几何代码　Makatura *et al*. [22]和Ko *et al*. [18]的研究探索了预训练模型（如GPT-3和GPT-4）以CSG代码片段的形式生成连贯的3D几何图形的潜力。

这些模型缺乏特定任务的微调，即使对于简单的任务，也经常产生不准确的几何输出。由于训练数据不足，它们有限的空间推理能力使精确的3D几何生成特别具有挑战性。

CAD序列生成　DeepCAD [46]、SkexGen [47]和HNC-CAD [48]等方法使用基于transformer的架构来生成CAD序列。DeepCAD主要执行两个功能：自动编码CAD设计和生成随机形状。虽然它们的用例有限，但它们展示了模型产生合理几何图形的能力。

Text2CAD [17]通过将自然语言转换为2D草图和3D操作，从文本描述生成CAD序列。该系统在其数据标注管道中使用LLMs和VLMs为CAD模型创建不同复杂度的文本提示，并采用基于transformer的自回归架构来生成构造序列。然而，该方法依赖于专业的CAD序列训练数据。此外，它不使用llm进行3D几何生成，并且不能在指定位置完成现有的3D几何。

构造立体几何　构造实体几何（CSG）是一种利用布尔运算将简单形状组合成更复杂形状的三维建模技术。这些形状可以是几何基元，如圆柱体、长方体、球体或由表面定义的细胞。尽管CSG模型只是一系列简单的操作，但它可以准确地表示复杂的形状。

从三维模型逆向工程CSG序列的过程涉及分析模型的几何特征和结构，将其分解为更简单的形状或表面。一些基于学习的方法试图从点云生成CSG树[6, 16, 31, 52]。CSGNet [35]也是一种基于学习

的方法，但是它使用体素网格作为3D几何的输入。这些基于学习的方法对于不精确的输入格式（如点云和体素网格）是必要的，但这也使它们不可靠。McCAD [13]和GEOUNED [3]是将BREP作为输入的规则库方法。这使他们能够遵循精确定义的边界表示几何，使用一组规则以可靠的方式生成准确的CSG表示。

3. 方法

Fig. 2Fig. 3分别显示了通过表示3D模型的脚本和该脚本的一个示例创建训练数据的管道。3D几何图形首先被转换成半空间（单元）序列，并在一个特定的点上分裂，这表示分裂为一个子部分几何图形和导致子部分完成的剩余几何图形。每个半空间由一组超平面和/或柱面定义，它们在Python脚本中表示为可重用的变量。与脚本生成过程并行，创建输出序列的2D渲染，然后由GPT-4o注释[28]。

产生的Python代码对，如Fig. 3所示，由作为LLM输入的3D模型的第一部分（Fig. 3a）和作为LLM目标输出的第二部分（Fig. 3b）组成。几何输入和输出各包含至少一个单元。除了3D模型之外，输入还包含在输出中重用的所有表面变量以及输出的文本描述。

3.1. 脚本生成

将三维几何图形转换为代码脚本需要以下步骤：模型分解、模型序列生成、输入输出分割、输入曲面确定和Python代码生成。

模型分解　CSG的生成可以通过基于学习或基于规则的方法进行。基于规则的方法因其更高的可靠性而被首选，但需要边界表示（BREP）格式的3D几何体。考虑到通常采用的BREP方法主要针对机械零件的生成，基于规则的方法适合于该方法。因此，选择GEOUNED [3]，一个为BREP到CSG转换而设计的工具，用于CSG生成。GEOUNED采用一种利用超平面和非平面边界的分解过程来创建单元（半空间），这些单元共同代表原始3D几何形状。如果一个单元格的体积是有限的，即它的任何一侧都没有打开，那么这个单元格就是有效的。分解阶段从导入STEP文件开始，然后使用GEOUNED分割它们。

模型序列生成　创建了该模型的图形，通过Fig. 2中的第二张图像进行可视化。这个图包含了模型作为节点组成的所有半空间。如果两个半空间在模型中相互接触，则它们在图中是连通的。该图用作生成合理序列的基础，以生成相同的3D几何图形，并避免生成不连接的几何图形。生成所有合理的序列以最大化训练数据量。一个看似合理的序列显示在Fig. 2的第三张图片中。

输入输出分离　为了准备训练数据，将每个序列分为模型输入分量和目标输出分量。序列在单元格之间的每个可能点进行分割，以最大化训练数据量。

输入表面测定　对于每个输入-输出划分，识别完成输入所需的表面。这些表面作为位置输入，以便模型可以学习在特定位置生成3D几何图形。

代码生成　在最后一步，创建的序列被转换为代码。OpenMC [32]是一个Python库，支持创建超平面和柱面，并将它们组合到单元格中。如Fig. 3a所示，输入的第一行显示了需要重用以创建输出几何图形的表面。脚本的其余部分将超平面和柱面定义为变量，并使用它们来生成单元格。

3.2. 几何补全模型

几何补全模型是走向文本控制生成模型的初步步骤。它是在类似于Fig. 3中显示的脚本上进行训练的，但缺少几何图形的文本描述。它的功能是接受一个不完整的3D几何图形作为输入，并通过重用输入中给定的曲面并创建新曲面来产生完整的几何图形作为输出。这是通过对代码生成模型DeepSeek-Coder-1b [11]进行微调来完成的。

3.3. 注释

文本注释描述了要生成的几何形状，以便模型根据文本描述学习完成几何形状。由于从3D几何自动生成文本描述仍未得到充分探索，我们使用GPT4o来注释2D图像，类似于Text2CAD方法[17]。这些图像是从不同的角度对3D几何图形进行简单的2D渲染。下面的提示用于生成几何特征的文本描述：

> 从四个不同的角度向你展示一个或一组零件。用一个或多个非常简短的非正式笔记描述零件的三维形状。不要提及不同的观点。尽量保持简短。不要写完整的句子。

我们的目标是保持注释尽可能简洁，并避免ChatGPT通常生成的冗长解释。

3.4. 文本控制的几何补全模型

文本控制的几何补全模型是几何补全的一个进步利用文本注释作为额外训练数据的补全模型，以便生成描述3D模型特征的文本指令。

4. 实验

4.1. 数据

我们选择在ABC数据集[19]上评估我们的方法，与Fusion360画廊[43]和Thingi10K [55]相反，它是公开可用的包含BREP格式机械零件的最大数据集。由于这项工作是概念验证，所以我们的目标是保持数据尽可能简单。因此，只从数据集中选择与基坐标系轴线垂直或平行的专用超平面和柱面可创建的部分。为了使部件的尺寸更均匀，我们删除了所有包含10个以上单元格的部件。由于我们训练一个补全模型，单元格的最小数量需要为两个：一个用于输入，一个用于输出。因此，我们也删除了所有只包含一个单元格的部

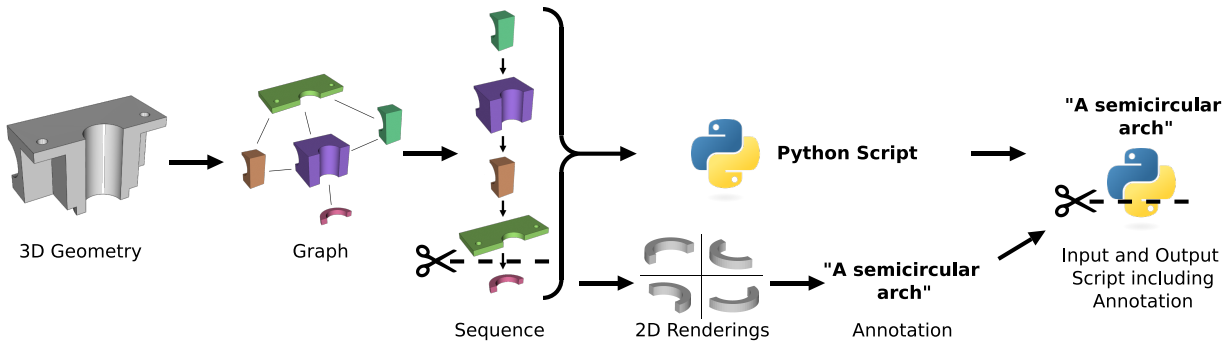


Figure 2. 数据生成流水线。首先将三维几何图形转换为图形，然后对其进行序列化。该序列分为输入和输出，并转换为python脚本。并行地，输出被渲染为2D图像并转换为文本注释，最后添加到脚本数据中。

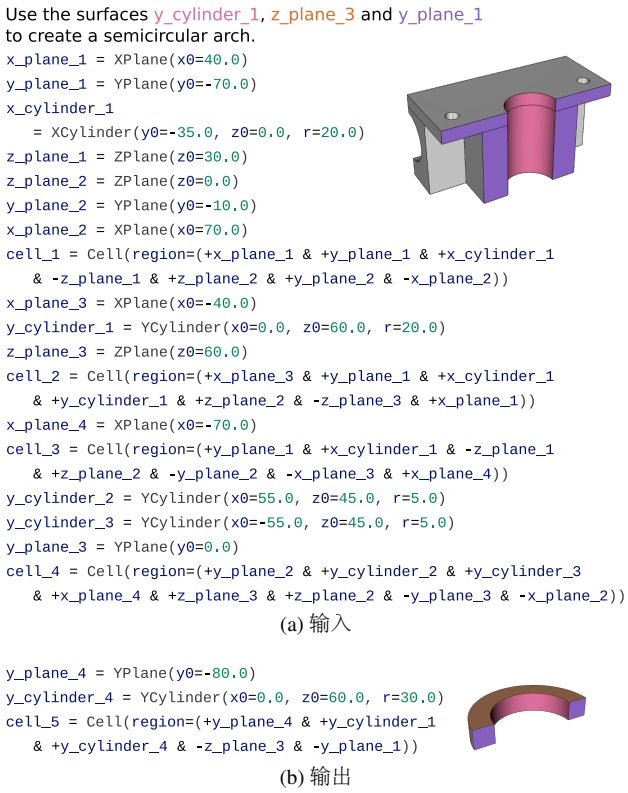


Figure 3. 表示3D几何图形的脚本示例。该脚本分为一个基础几何和额外的部分，应该学习几何完成。

分。零件尺寸分布见Fig. 4。此外，我们从数据集中删除了所有重复的数据，包括那些在缩放、旋转或平移方面存在差异的数据。我们得到的数据集包含37220个部件。在90/10的训练/测试分割中，我们有33498个部件用于训练。在我们的实验中，我们使用两种数据增强方法来增加这个数据集的大小。首先，我们改变序列被切割为输入和输出的位置，其次我们改变序列的顺序。

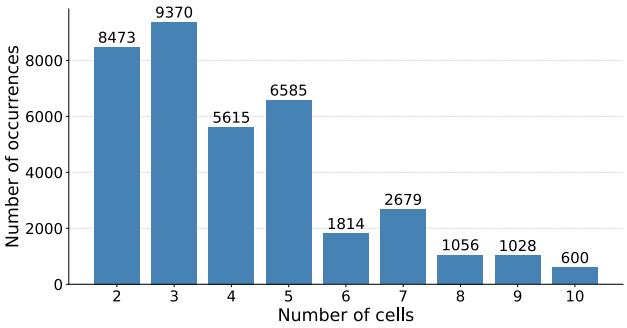


Figure 4. 精选数据集中按单元格数量划分的部件分布。

4.2. 评价策略

据我们所知，目前还没有CAD几何图形完成的可比模型。DeepCAD [46]和Text2CAD [17]无法在给定位置完成现有的几何图形。因此，该模型被自动评估了生成几何图形的合理性，并手动评估了与自然语言输入相关的几何图形的正确性。此外，对模型本身的版本进行评估，以证明文本描述增加了模型达到真实值的可能性。

4.3. 几何补全模型

几何补全模型是最终模型的简化版本，不包括文本注释。此评估的目的是评估模型仅根据输入几何图形生成合理输出几何图形的能力。Tab. 1表明，改变割会降低模型的性能，而改变序列的顺序会提高模型在所有参数上的性能。理想情况下，显示Python代码的语法正确性、单元格的连通性、重叠单元格的无重叠、语法和逻辑的组合正确性以及所有指示表面的利用率应尽可能接近1.0。剩余的参数显示在Tab. 1的最后三行中，显示了模型生成与地面真实值相似或相同的3D几何图形的能力，然而，生成与地面真实值不同的几何图形并不被认为是错误的，因为有接近无限个正确的解决方案。因此，这些参数的绝对值是不相关的，但这些参数对于不同的增强策略的差异是很有意义的。

简而言之，Tab. 1表明，该模型能够在绝大多数情况

Variation Method	None	Cut	Order	Cut & Order
Training Data Size	33,498	101,371	140,571	550,787
Training Time (hours)	2.2	6.5	9.0	35.2
Correct Syntax	0.999	0.960	1.0	0.981
All Cells Connected	0.977	0.954	0.980	0.966
No Overlapping Cells	0.989	0.980	0.993	0.979
Correct Syntax & Logic	**0.965**	**0.895**	**0.973**	**0.927**
All Surfaces Used	0.741	0.738	0.830	0.822
Same Number of Cells	0.494	0.439	0.580	0.565
Same Syntax, Diff. Params	0.105	0.100	0.159	0.167
Same Syntax, Same Params	0.050	0.042	0.067	0.064

Table 1. 模型性能比较。一个几何图形在逻辑和语法上是正确的当语法正确时，所有单元格都是连接的，没有重叠。

下生成逻辑上和语法上正确的3D几何图形，并通过对序列进行不同的排序来增加训练数据，从而提高模型的性能。

除其他外，Tab. 1显示生成的几何图形有多少份额的单元格数量等于地面真相中的单元格数量。Fig. 5更详细地探讨了这个问题。矩阵的每行表示测试数据集中对应的真实值中特定数量的单元格生成的单元格数量的分布。例如，矩阵的最低一行表示所有输入输出对，其中真实值输出只包含一个单元格。在84%的情况下，实际生成的细胞数也是1。

图中显示，对于几乎任何数量的真值单元，生成的最常见的细胞数量等于真值中的细胞数量。这表明模型从输入的几何形状以及选择的曲面来理解结果几何形状应该有多复杂。随着输出单元数的增加，分布曲线变得更平坦。这并不意味着模型在预测更复杂几何体的单元数方面更差。自然地，随着自然数的增加，预测正确数字的概率会降低，因此可以预期一个更平坦的分布。对于ground truth cell count为9的情况，样本量只有5，这使得它太小，无法判断模型。由于细胞总数在10处被切断，并且输入至少为1个细胞，因此在训练数据中，目标输出从未超过9个细胞。在实践中，该模型仅在非常罕见的情况下产生了9个以上的细胞。

Fig. 6 与输入单元格的数量和输出单元格的数量相比，显示了在语法上与真实值相同的结果的份额。部件只检查相同的结构，这意味着参数的差异，从而在比例部分的差异是可能的。例如，矩阵第一列的第一行的条目显示，对于包含9个单元格的输入和包含1个单元格的基准真值输出，其中34%的输入在语法上与基准真值相同。生成结果与输出相等的份额较低并不意味着模型是错误的，因为有无限个可能的解决方案，而不仅仅是基本事实的输出。然而，与真实值相当的较高百分比表明模型更善于理解如何在测试数据集中生成与人类设计的结构相似的结构。

该矩阵表明，随着真实值中单元格数量的增加，生成的输出不太可能等于真实值。这是因为它在概率上不太可能预测复杂结构。输入单元格数对精度的影响不明显。似乎输入单元格的数量越多，生成等于真实值的输出的机会就越大。对此的一种可能解释是，细胞数越少的输入越不明确，因此留给进一步发展的空间就越大。另一方面，如果输入已经包含特定的模式，则模型可能会继续该模式。

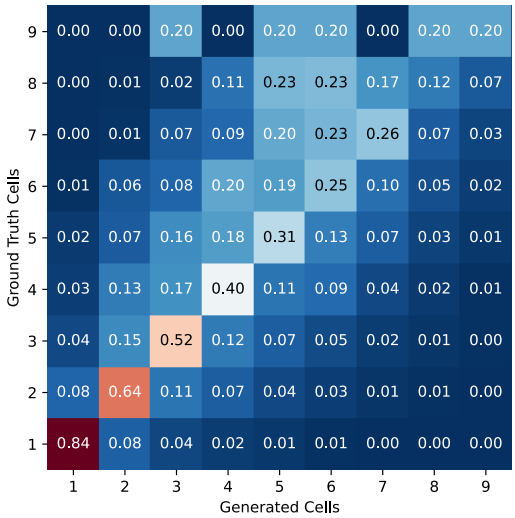


Figure 5. 几何补全模型为每个真值单元数生成的单元数的分布。每行加起来等于1，并显示所有元素生成的单元格数量的分布，这些元素在它们的真实值中具有一定数量的单元格。

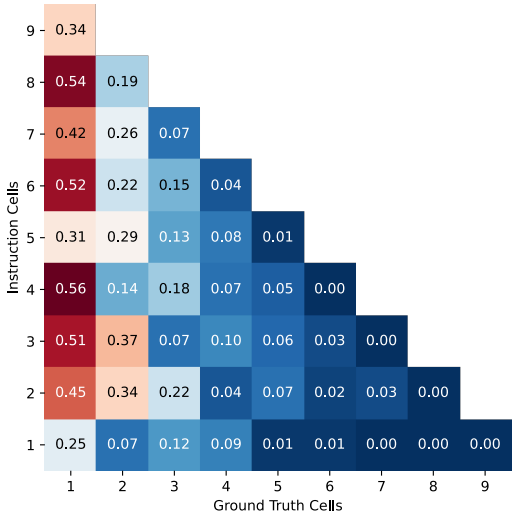


Figure 6. 根据输入单元的数量和真值单元的数量，生成的输出与真值相等（可能的不同参数）的结果的一部分。

4.4. 文本注释

Tab. 2 显示生成的文本注释的准确性。对于每一个单元格数量，人工评估了100个文本注释，并将其分类为以下类别之一：

- 正确 文本描述是百分之百正确的，并描述了该部分的整体。
- 部分纠正 文本的描述在某一方面是错误的，或者没有完整地描述该部分。错误的例子是错误的计数或将一个长而细的矩形梁误认为一个长而细的圆柱体。
- 错误 文本描述大部分或完全不正确。

Tab. 2 结果表明，对于两个以上单元格的几何图形，文本标注的准确性明显下降。由于在Sec. 4.1中显示的具有较少单元格数量的几何图形的过度表示，文本注释的平均精度为80%。复杂几何图形的文本注释质量较低这一事实表明，与简单几何图形相比，在此数据上训练的模型在生成更复杂的几何图形方面可能更差。一些注释数据质量低的原因之一是，一些几何图形很难用单词描述，即使是人类也很难。当几何图形很复杂或者是另一个几何图形的一部分，很难单独理解时，这一点尤其正确。

Number of Cells	1	2	3	4	5	6	7	8	9
Correct	0.92	0.81	0.63	0.54	0.50	0.41	0.30	0.35	0.39
Partially Correct	0.07	0.13	0.31	0.38	0.41	0.51	0.60	0.56	0.52
Incorrect	0.01	0.06	0.06	0.08	0.09	0.08	0.10	0.09	0.09

Table 2. 文本注释的准确性，由单元格的数量决定。

4.5. 文本控制的几何补全模型

Tab. 3 显示与输出单元格数量相关的等于真实值的生成部分的份额。在输入中包含输出的文本注释，增加了生成等于真实值的输出的可能性，从而表明可以通过文本输入控制输出的几何形状。Tab. 3表示对于1和2的单元格计数是这样的。对于具有两个以上单元格的输出，文本注释不会增加生成与真实值相等的输出的可能性。Sec. 4.4表明，对于超过2个单元格的几何图形，文本注释质量显著下降。缺乏高质量数据解释了为什么如果输出有2个以上单元格，文本注释不能帮助模型生成更接近真实的输出。

Ground Truth Cells	1	2	3	4	5	6
Without Text Input	0.45	0.22	0.14	0.06	0.02	0.01
With Text Input	0.51	0.25	0.14	0.06	0.02	0.01

Table 3. 根据生成的输出等于地面真值（可能不同的参数）的结果比较两个模型，按地面真值单元的数量。

Fig. 7 展示了一些实际生成的3D模型。第一列包含输入几何图形的渲染，包括作为输入给出的表面。第二列显示了ground truth的文本描述，这是由GPT4o生成的。在第三列和第四列中，真实值以及模型生成的结果与输入的几何图形（灰色）相结合显示（绿色）。

第一行显示了一个生成的两个单元格在形状和参数方面与真实值相等的示例。第二行、第三行和第四行展示了真实值和生成的几何图形在形状方面相等，但在参数方面不同的例子。第五行展示了一个例子，该模型遵循表面输入，并以合理的方式完成了几何图形，而没有遵循自然语言输入。基本事实由六个单元格组成，这种复杂性使模型难以遵循自然语言指令。

在最后两行中，输入的几何图形是同一个玩具砖的不同的不完整变体。在第一个示例中，模型未能准确遵循文本描述。在第二个示例中，文本描述是模糊的，模型准确地遵循它，即使它与真实情况不同。值

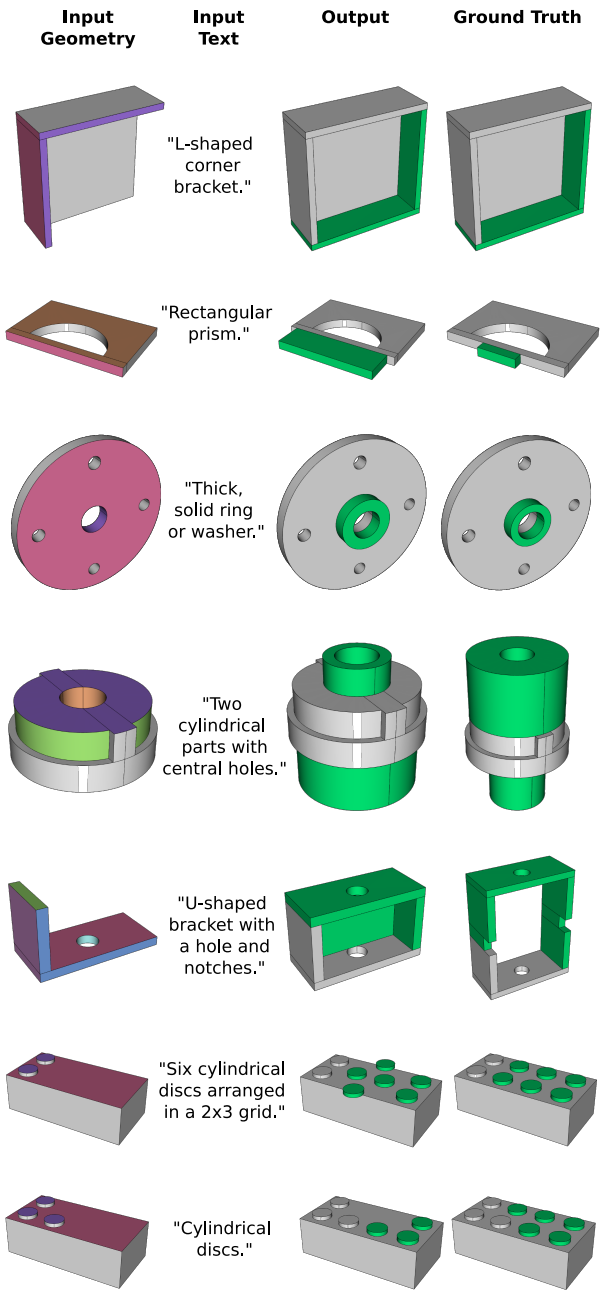


Figure 7. 定性结果。在前4个实验中，模型准确地遵循了文本指令。在最后3个实验中，模型生成了合理的几何图形，但它与文本指令不一致。

得注意的是，通常模型试图创建一个对称部分，可能是因为训练数据中有许多对称部分。

模型遵循文本指令的能力总结在Tab. 4中。该表是通过手动评估每个单元格数量的测试数据中的50个随机样本创建的。实验结果表明，随着生成单元格数量的增加，模型的准确率显著下降。这是由于更复杂的几

何图形的注释数据质量较低，如Sec. 4.4所示。

总而言之，实验表明，该模型能够学习3D几何，并能够生成语法和数学上正确且可信的结果。此外，只要要生成的几何图形不是太复杂，它就能够跟踪位置输入和文本输入。假设有了更高质量的文本注释，模型遵循文本指令生成复杂几何图形的能力将会提高。

Number of Cells	1	2	3	4	5	6	7
Correct	0.82	0.57	0.34	0.31	0.29	0.23	0.19
Partially Correct	0.11	0.33	0.52	0.43	0.37	0.46	0.54
Incorrect	0.07	0.10	0.14	0.26	0.34	0.31	0.27

Table 4. 模型遵循文本指令的准确性，由单元格数量决定。

5. 讨论与结论

讨论。 所提出方法展示了一个有希望的CSG生成管道，尽管它面临一些限制。为简单起见，模型仅在由垂直或平行于基础坐标系轴线的平面和圆柱体组成的3D几何图形上进行训练。未来的工作可以探索生成更高级的几何图形，可能通过利用更大的代码生成LLM。此外，模型遵循文本指令的能力仍然有限。一个关键的挑战是，即使是简单的抽象形状也很难用语言描述，尽管矛盾的是，复杂的几何形状或操作有时可以用简单的单词来表达。因此，我们相信用于CAD的自然语言输入将补充而不是取代传统的CAD方法。例如，当修改一个已有的零件时，口头描述修改的位置是不必要的复杂，而直接选择表面则更加直观。虽然我们当前的模型只能扩展现有的几何形状，但修改后的数据管道可以修改现有的结构。这些修改通常口头上描述得很简单，但实施起来很耗时，例如改变齿轮上的齿数或调整外壳尺寸。

结论。 提出了一种生成3D几何的新方法，利用代码生成LLM来生成基于表面的构造性实体几何（CSG）。我们的管道首先将边界表示几何（BREP）转换为基于csg的Python脚本，创建一个表示为代码的3D机械零件数据集。然后，我们使用GPT-4生成自然语言注释，并使用此数据集对代码生成LLM进行微调。该方法既是第一个专门为生成CSG几何而训练的深度学习模型，也是第一个能够基于文本指令完成3D几何的微调LLM。数据管道的使用。STEP文件作为输入确保了跨CAD软件平台的兼容性。实验表明，该模型成功地学习了3D几何，生成了语法和数学上合理的结果。此外，该模型有效地遵循基本几何结构的位置和文本输入。

References

[1] Andrew Brock, Theodore Lim, J. M. Ritchie, and Nick Weston. Generative and discriminative voxel modeling with convolutional neural networks, 2016. 2

[2] Ruojin Cai, Guandao Yang, Hadar Averbuch-Elor, Zekun Hao, Serge Belongie, Noah Snavely, and Bharath Hariharan. Learning gradient fields for shape generation, 2020. 2

[3] J.P. Catalán, P. Sauvan, J. García, J. Alguacil, F. Ogando, and J. Sanz. Geouned: A new conversion tool from cad to monte carlo geometry. *Nuclear Engineering and Technology*, 56(6): 2404–2411, 2024. 3

[4] Zhiqin Chen, Andrea Tagliasacchi, and Hao Zhang. Bspnet: Generating compact meshes via binary space partitioning, 2020. 2

[5] Christopher B. Choy, Danfei Xu, JunYoung Gwak, Kevin Chen, and Silvio Savarese. 3d-r2n2: A unified approach for single and multi-view 3d object reconstruction, 2016. 2

[6] Markus Friedrich, Pierre-Alain Fayolle, Thomas Gabor, and Claudia Linnhoff-Popien. Optimizing evolutionary csg tree extraction. In *Proceedings of the Genetic and Evolutionary Computation Conference*, page 1183–1191, New York, NY, USA, 2019. Association for Computing Machinery. 2

[7] Rao Fu, Xiao Zhan, Yiwen Chen, Daniel Ritchie, and Srinath Sridhar. Shapecrafter: A recursive text-conditioned 3d shape generation model, 2023. 2

[8] Kyle Genova, Forrester Cole, Daniel Vlasic, Aaron Sarna, William T. Freeman, and Thomas Funkhouser. Learning shape templates with structured implicit functions, 2019. 2

[9] GitHub. Copilot. https://github.com/features/copilot, 2024. Accessed: 2024-08-04. 1

[10] Thibault Groueix, Matthew Fisher, Vladimir G. Kim, Bryan C. Russell, and Mathieu Aubry. Atlasnet: A papier-mâché approach to learning 3d surface generation, 2018. 2

[11] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. Deepseek-coder: When the large language model meets programming – the rise of code intelligence, 2024. 3

[12] Rana Hanocka, Amir Hertz, Noa Fish, Raja Giryes, Shachar Fleishman, and Daniel Cohen-Or. Meshcnn: a network with an edge. *ACM Transactions on Graphics*, 38(4):1–12, 2019. 2

[13] Moataz Harb, Dieter Leichtle, and Ulrich Fischer. A novel algorithm for cad to csg conversion in mccad. *Journal of Nuclear Engineering*, 4(2):436–447, 2023. 3

[14] Krystal Hu. Chatgpt sets record for fastest-growing user base - analyst note | reuters, 2023. 1

[15] R. Kenny Jones, Theresa Barton, Xianghao Xu, Kai Wang, Ellen Jiang, Paul Guerrero, Niloy J. Mitra, and Daniel Ritchie. Shapeassembly: learning to generate programs for 3d shape structure synthesis. *ACM Transactions on Graphics*, 39(6):1–20, 2020. 2

[16] Kacper Kania, Maciej Zieba, and Tomasz Kajdanowicz. Ucsg-net- unsupervised discovering of constructive solid geometry tree. In *Advances in Neural Information Processing Systems*, pages 8776–8786. Curran Associates, Inc., 2020. 2

[17] Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2cad: Generating sequential cad models from beginner-to-expert level text prompts, 2024. 1, 2, 3, 4

[18] Jaechang Ko, John Ajibefun, and Wei Yan. Experiments on generative ai-powered parametric modeling and bim for architectural design, 2023. 2

[19] Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. Abc: A big cad model dataset for geometric deep learning. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. 3

[20] Zhengzhe Liu, Yi Wang, Xiaojuan Qi, and Chi-Wing Fu. Towards implicit text-guided 3d shape generation, 2022. 2

[21] Shitong Luo and Wei Hu. Diffusion probabilistic models for 3d point cloud generation, 2021. 2

[22] Liane Makatura, Michael Foshey, Bohan Wang, Felix Hähn-Lein, Pingchuan Ma, Bolei Deng, Megan Tjandrasuwita, Andrew Spielberg, Crystal Elaine Owens, Peter Yichen Chen, Allan Zhao, Amy Zhu, Wil J Norton, Edward Gu, Joshua Jacob, Yifei Li, Adriana Schulz, and Wojciech Matusik. How can large language models help humans in design and manufacturing?, 2023. 2

[23] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis, 2020. 2

[24] Charlie Nash, Yaroslav Ganin, S. M. Ali Eslami, and Peter W. Battaglia. Polygen: An autoregressive generative model of 3d meshes, 2020. 2

[25] Alex Nichol, Heewoo Jun, Prafulla Dhariwal, Pamela Mishkin, and Mark Chen. Point-e: A system for generating 3d point clouds from complex prompts, 2022. 2

[26] Onshape. Onshape: Cloud-native product development platform. https://www.onshape.com, n.d. Accessed: 2024-11-11. 1

[27] OpenAI. Chatgpt: A large-scale generative model for open-domain chat. https://openai.com/blog/chatgpt, 2022. 1

[28] OpenAI. Gpt-4v(ision) system card, 2023. 3

[29] Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. Deepsdf: Learning continuous signed distance functions for shape representation, 2019. 2

[30] Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion, 2022. 1, 2

[31] Daxuan Ren, Jianmin Zheng, Jianfei Cai, Jiatong Li, Haiyong Jiang, Zhongang Cai, Junzhe Zhang, Liang Pan, Mingyuan Zhang, Haiyu Zhao, and Shuai Yi. Csg-stump: A learning friendly csg-like representation for interpretable shape parsing, 2021. 2

[32] Paul K. Romano, Nicholas E. Horelik, Bryan R. Herman, Adam G. Nelson, Benoit Forget, and Kord Smith. Openmc: A state-of-the-art monte carlo code for research and development. *Annals of Nuclear Energy*, 82:90–97, 2015. Joint

International Conference on Supercomputing in Nuclear Applications and Monte Carlo 2013, SNA + MC 2013. Pluri- and Trans-disciplinarity, Towards New Modeling and Numerical Simulation Paradigms. 3

[33] Aditya Sanghi, Hang Chu, Joseph G. Lambourne, Ye Wang, Chin-Yi Cheng, Marco Fumero, and Kamal Rahimi Malekshan. Clip-forge: Towards zero-shot text-to-shape generation, 2022. 2

[] Raoul Schönhof, Jannes Elstner, Radu Manea, Steffen Tauber, Ramez Awad, and Marco F. Huber. Simplified learning of cad features leveraging a deep residual autoencoder, 2022.

[35] Gopal Sharma, Rishabh Goyal, Difan Liu, Evangelos Kalogerakis, and Subhransu Maji. Csgnet: Neural shape parser for constructive solid geometry, 2018. 2

[36] Dong Wook Shu, Sung Woo Park, and Junseok Kwon. 3d point cloud generative adversarial network based on tree structured graph convolutions, 2019. 2

[37] Yawar Siddiqui, Antonio Alliegro, Alexey Artemov, Tatiana Tommasi, Daniele Sirigatti, Vladislav Rosov, Angela Dai, and Matthias Nießner. Meshgpt: Generating triangle meshes with decoder-only transformers. *arXiv preprint arXiv:2311.15475*, 2023. 1, 2

[38] Vincent Sitzmann, Michael Zollhöfer, and Gordon Wetzstein. Scene representation networks: Continuous 3d-structure-aware neural scene representations, 2020. 2

[39] Stack Overflow. 2024 Stack Overflow Developer Survey, 2024. 1

[40] Maxim Tatarchenko, Alexey Dosovitskiy, and Thomas Brox. Octree generating networks: Efficient convolutional architectures for high-resolution 3d outputs, 2017. 2

[41] Nobuyuki Umetani. Exploring generative 3d shapes using autoencoder networks. In *SIGGRAPH Asia 2017 Technical Briefs*, New York, NY, USA, 2017. Association for Computing Machinery. 2

[42] Diego Valsesia, Giulia Fracastoro, and Enrico Magli. Learning localized generative models for 3d point clouds via graph convolution. In *International Conference on Learning Representations*, 2019. 2

[43] Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences, 2021. 3

[44] Jiajun Wu, Chengkai Zhang, Tianfan Xue, William T. Freeman, and Joshua B. Tenenbaum. Learning a probabilistic latent space of object shapes via 3d generative-adversarial modeling, 2017. 2

[45] Jiajun Wu, Chengkai Zhang, Tianfan Xue, William T. Freeman, and Joshua B. Tenenbaum. Learning a probabilistic latent space of object shapes via 3d generative-adversarial modeling, 2017. 2

[46] Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models, 2021. 1, 2, 4

[47] Xiang Xu, Karl DD Willis, Joseph G Lambourne, Chin-Yi Cheng, Pradeep Kumar Jayaraman, and Yasutaka Furukawa. Skexgen: Autoregressive generation of cad construction sequences with disentangled codebooks. In *International Conference on Machine Learning*, pages 24698–24724. PMLR, 2022. 2

[48] Xiang Xu, Pradeep Kumar Jayaraman, Joseph G. Lambourne, Karl D. D. Willis, and Yasutaka Furukawa. Hierarchical neural coding for controllable cad model generation, 2023. 2

[49] Xiang Xu, Joseph G. Lambourne, Pradeep Kumar Jayaraman, Zhengqing Wang, Karl D. D. Willis, and Yasutaka Furukawa. Brepgen: A b-rep generative diffusion model with structured latent geometry, 2024. 2

[50] Guandao Yang, Xun Huang, Zekun Hao, Ming-Yu Liu, Serge Belongie, and Bharath Hariharan. Pointflow: 3d point cloud generation with continuous normalizing flows, 2019. 2

[51] Yaoqing Yang, Chen Feng, Yiru Shen, and Dong Tian. Foldingnet: Point cloud auto-encoder via deep grid deformation, 2018. 2

[52] Fenggen Yu, Zhiqin Chen, Manyi Li, Aditya Sanghi, Hooman Shayani, Ali Mahdavi-Amiri, and Hao Zhang. Capri-net: Learning compact cad shapes with adaptive primitive assembly, 2021. 2

[53] Maciej Zamorski, Maciej Zięba, Piotr Klukowski, Rafał Nowak, Karol Kurach, Wojciech Stokowiec, and Tomasz Trzciński. Adversarial autoencoders for compact representations of 3d point clouds, 2019. 2

[54] Linqi Zhou, Yilun Du, and Jiajun Wu. 3d shape generation and completion through point-voxel diffusion, 2021. 2

[55] Qingnan Zhou and Alec Jacobson. Thingi10k: A dataset of 10,000 3d-printing models, 2016. 3